

The Best MCU Peripheral You've Never Heard Of

Nathan Jones

The Best MCU Peripheral You've Never Heard Of

- Introduction

- What is it?

- What can you do with it?

- Manchester encoding

- Emergency shut-off

- 16-bit DAC

- Laser tag

- And so much more...

- Digital logic for all!

- Cypress PSoC

- Texas Instruments C2000

- Field-programmable Gate Array (FPGA)

- Complex programmable logic device (CPLD)

- The design space grows

- Conclusion

- Solutions

- Priority encoder

- Toaster FSM

- Motor control

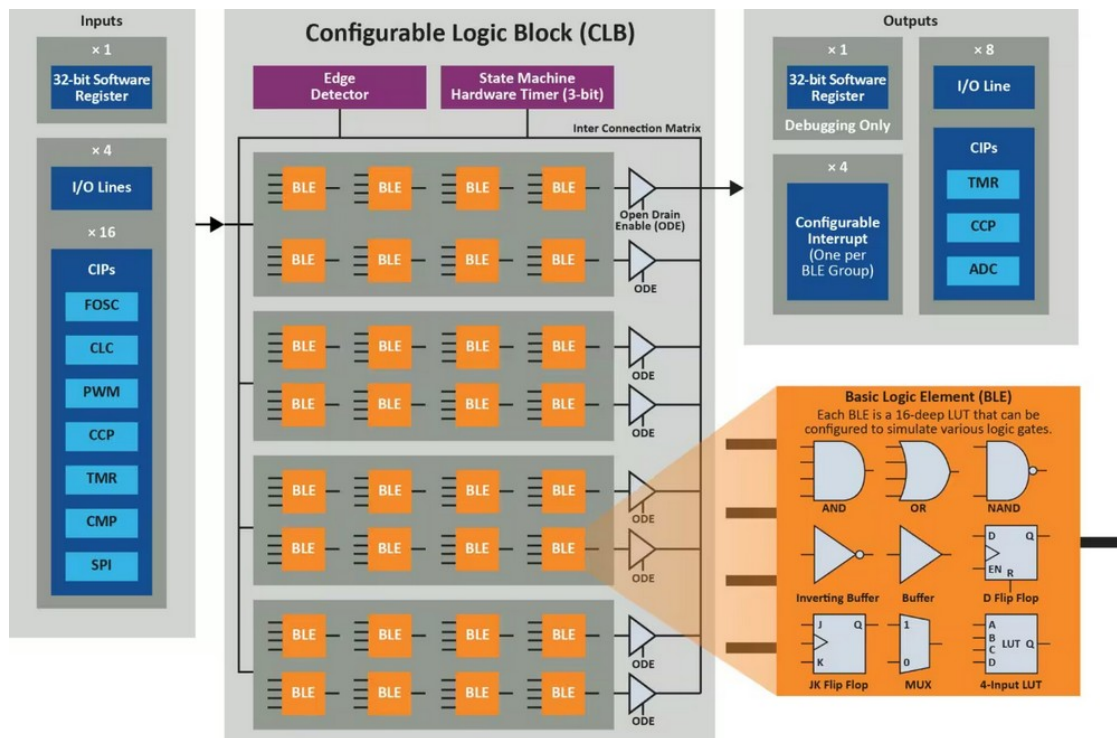
- Smart sensor

Introduction

Microcontrollers have tons of useful peripherals; GPIO, timers, UART, SPI, and I2C are such useful peripherals that they come on nearly every microcontroller these days. Niche peripherals like LCD drivers, internal op-amps, or CAN transceivers can, on the right project, reduce cost, complexity and component count. But none, IMHO, are quite as versatile as the **configurable logic block** (CLB) from Microchip.

What is it?

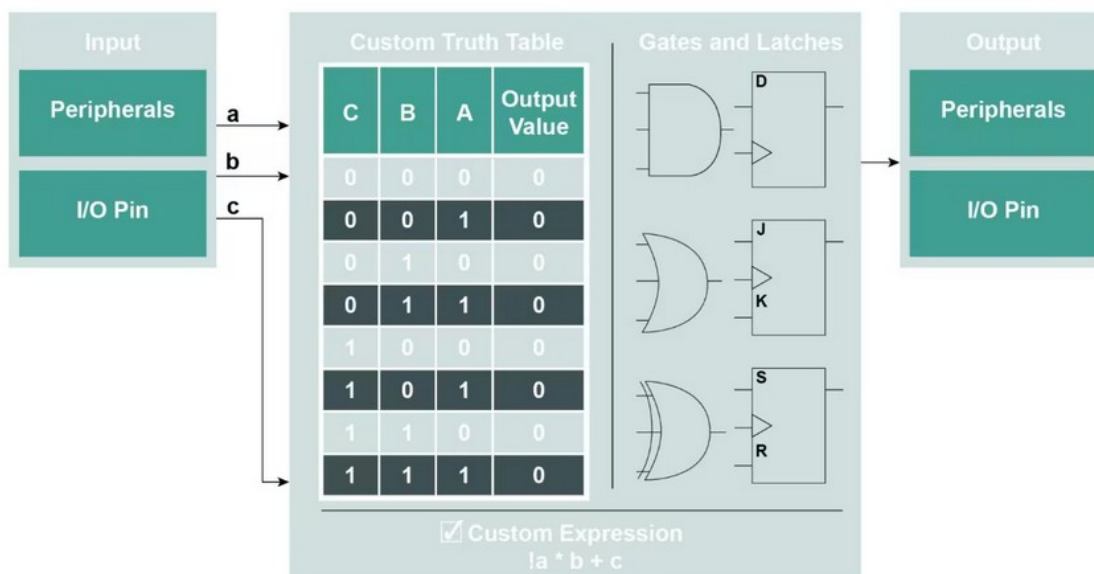
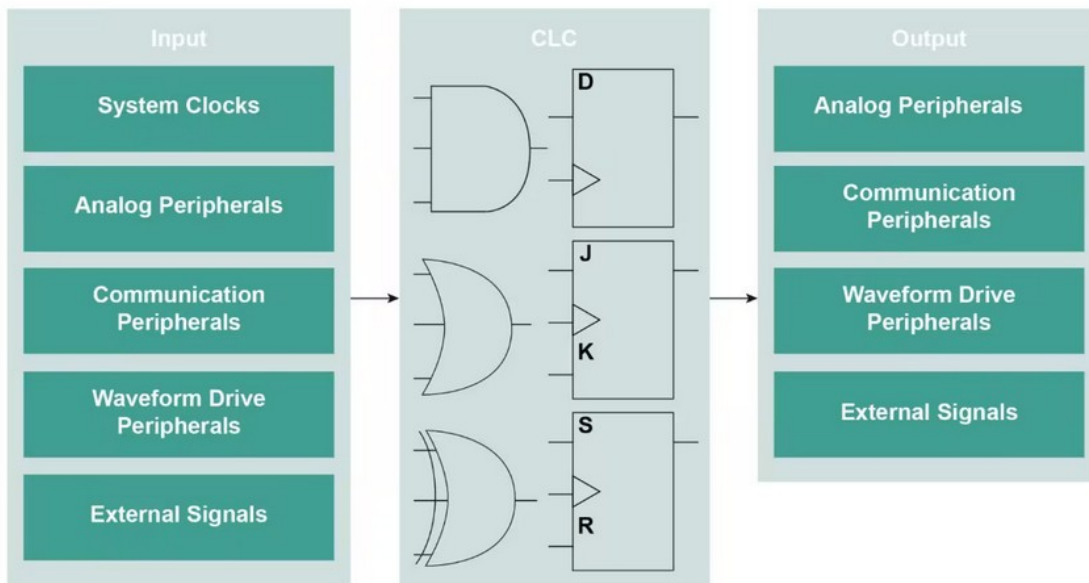
The CLB from Microchip (found on PIC16F131xx devices) is an example of an MCU peripheral that gives the designer a small piece of configurable digital logic inside their microcontroller.



From [Microchip, Custom Logic Peripherals](#)

The CLB contains a handful of logic elements (not unlike the dozens or thousands of logic elements that make up the internal fabric of an FPGA) that each implement a specific logic device: an AND gate, a multiplexer, a D flip-flop, a JK flip-flop, etc. The inputs to these blocks can come from a number of different signal sources inside the microcontroller (e.g. from a GPIO pin, from an internal timer, etc) and the outputs can also go to many different signal sinks (e.g. to a GPIO pin, to an interrupt signal, to another CLB logic element, etc).

Microchip also offers a peripheral called the **configurable logic cell (CLC)** on most (all?) of their PIC16F and PIC18F devices (along with two PIC10s and one PIC12) and another called the **configurable custom logic (CCL)** on most (all?) of their AVR devices.



These peripherals act exceedingly similarly to the CLB, with slight differences in which logic devices are supported (notice that the CLC and

CCL can each implement an SR latch, which the CLB cannot) and also where the inputs and outputs can be routed from/to.

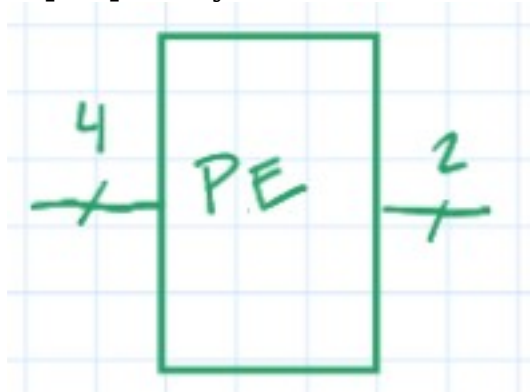
Although simple in their design, these peripherals can punch *way* above their weight class, helping to create highly complex systems that shift some of the software load or component count of a project onto small digital logic circuits.

We're about to see the power of digital logic, so let's dust off those digital logic design skills!

Challenge Question

Can you create digital designs that implement the following functionalities?

1. 4-input priority encoder ([solution](#))

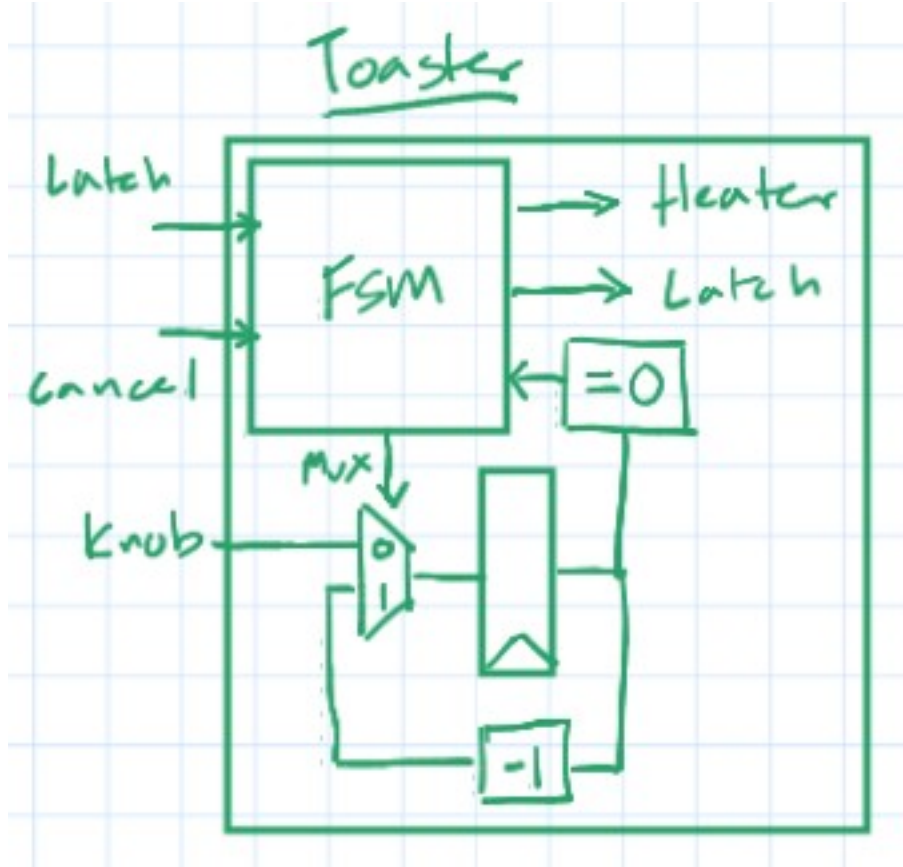


1. Takes 4 1-bit inputs and outputs a binary number corresponding to the highest input with an active bit.
2. Ex: If the input is 0110, the output will be 10 ("bit position 2 is active (and supersedes bit position 1)")
3. Hints
 1. A Karnaugh map will help you determine the Boolean expressions corresponding to each output.
 2. Did you realize that the output will need at least one additional bit?? Otherwise how would one differentiate between inputs of 0001 ("input 0 is active") and 0000 ("no input is active")?
4. Going further
 1. Add a fifth input to enable/disable the outputs of the priority encoder. What needs to change on the output now that the device has this additional mode of operation?
 2. How could you modify or extend your design to handle N bits of input? Can you do so without needing to design

a brand new circuit (i.e. can you compose an N-bit PE from some number of 4-bit PEs)? What are the trade-offs in your design, considering things like complexity and propagation delays?

1. Turns out, this is an [area of on-going research](#), too, not just an academic exercise!

2. Toaster FSM ([solution](#))



1. Operation

1. User sets the desired toastiness using up/down buttons, which set the value in a 4-bit down-counter used for timing purposes.
2. Toasting begins when the user presses down the latch. During toasting, the heating element should be on, the latch should be active (to hold down the toast), and the down-counter is active.
3. Toasting ends when the down-counter reaches 0 OR the user presses the cancel button.

2. Hints

1. Start by drawing the state diagram for our system. Ensure that *all* possible transitions are depicted.

2. Using the state diagram, fill in a state table that describes the next state values for each combination of current state and inputs. Next, complete an output table showing the outputs of the FSM for each of its states.
3. Draw the Karnaugh maps for each bit of the next state and outputs and determine the Boolean expressions for each of them.
4. Ensure you're design correctly handles ALL input combinations. For instance, what will your design do if:
 1. the latch is depressed while the cancel button is being held?
 2. the knob has been set to 0 when the latch is depressed?
 3. the latch is held down once the counter reaches 0 and the FSM transitions to its idle/waiting state?
3. Going further
 1. Update the design so that the FSM updates at 100 Hz (for fast UI responsiveness), but the down-counter operates at 1 Hz.
 2. Add a "bagel" button that will only turn on the outside heaters when the toaster is started.
 3. Add over-current or over-temperature sensors.
 4. Change the knob/counter so that the user can adjust the "toastiness" value while the toaster is on. E.g. raising the knob will increase the amount of time the toaster is heating. Lowering the knob will decrease that amount of time; if the user lowers the knob past however long the toaster has already been on, the toaster will immediately return to the "idle" state.

What can you do with it?

A handful of gates and flip-flops might not seem like much, but there is lots that they can do to help offload computation load or decrease component count in a final design. Here are some simple applications.

Manchester encoding

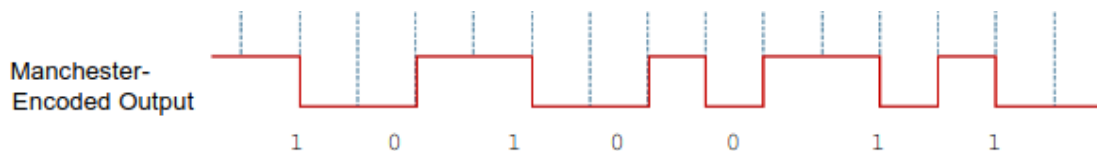
(From Microchip document DS41631B, "[Configurable Logic Cell Tips 'n Tricks](#)")

A "normal" bitstream will represent 1s and 0s as static signal levels. Consider a TTL-UART signal: 1s are 3.3/5V and 0s are 0V. The value

1010011 (MSB-first) would look like this (where each column represents half of the clock period):

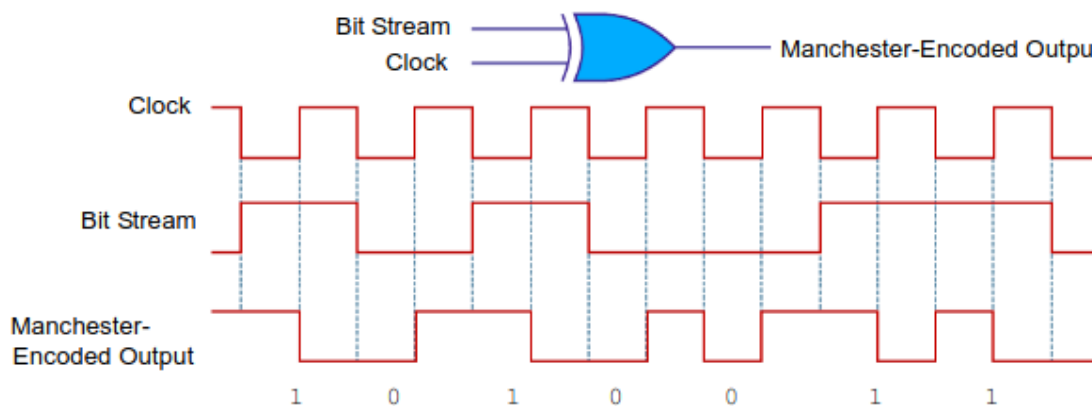


A Manchester-encoded signal, however, represents 0s as **low-to-high** transitions and 1s as **high-to-low** transitions (using traditional encoding; IEEE 802.3 associates 1s and 0s in the opposite manner, which makes a little more sense to me). The same value as before, when Manchester-encoded, would look like this:



Manchester-encoded signals have the benefit of being **self-clocking** and also have **no DC signal bias**, allowing the signal to be AC-coupled. These features are preferred for communications requiring only a single transmission channel (such as wireless communications) or that operate in noisy environments.

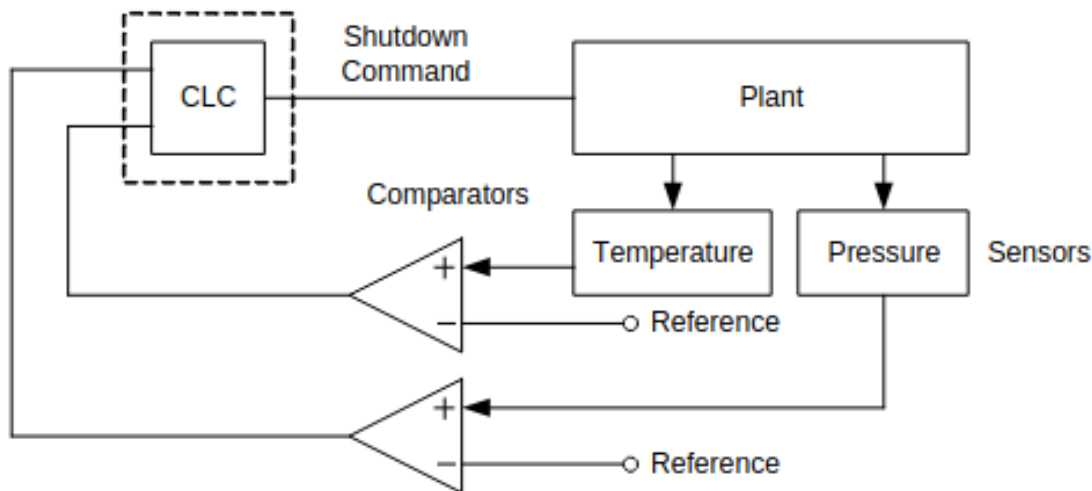
We can use a CLB to convert a clock and bitstream into a Manchester-encoded signal using just an XOR gate.



Emergency shut-off

(From Microchip appnote AN2133, ["Extending PIC MCU Capabilities Using CLC"](#))

An emergency shut-off is a necessary precaution for systems that are controlling heavy machinery (or, really, anything that could cause human or collateral damage if misused).



For example, by feeding the outputs of two analog comparators into the CLC above, we can detect when temperature or pressure is above or below a safe threshold and so generate a shutdown command. Using the CLC, we can combine these signals in a number of ways. The shutdown command generated by the CLC could be set by ANDing the comparators, by ORing them, or it could even be configured as the output of an SR latch which is set by an ORing of the two comparators, which would result in the shutdown command being held high even if the temperature and pressure returned to normal levels. This could ensure that the system remains off until a human operator determines that it can be restarted safely, by resetting the SR latch. For more complex behaviors, you could even construct an FSM inside the CLC (see [here](#) for an example FSM design using the CLC).

Because this is done **completely in hardware**, it stands a much better chance of passing a safety-critical verification than if the emergency shut-off were detected via software.

But the fun doesn't stop there! Here are a few more complex applications for the CLB.

16-bit DAC

This one comes from [a contest](#) that Microchip held earlier this year to find the most creative use of the CLB. Using all 32 elements in the CLB on a PIC16F13145, Mark Omo built a DAC that had an amazing 480 Hz BW,

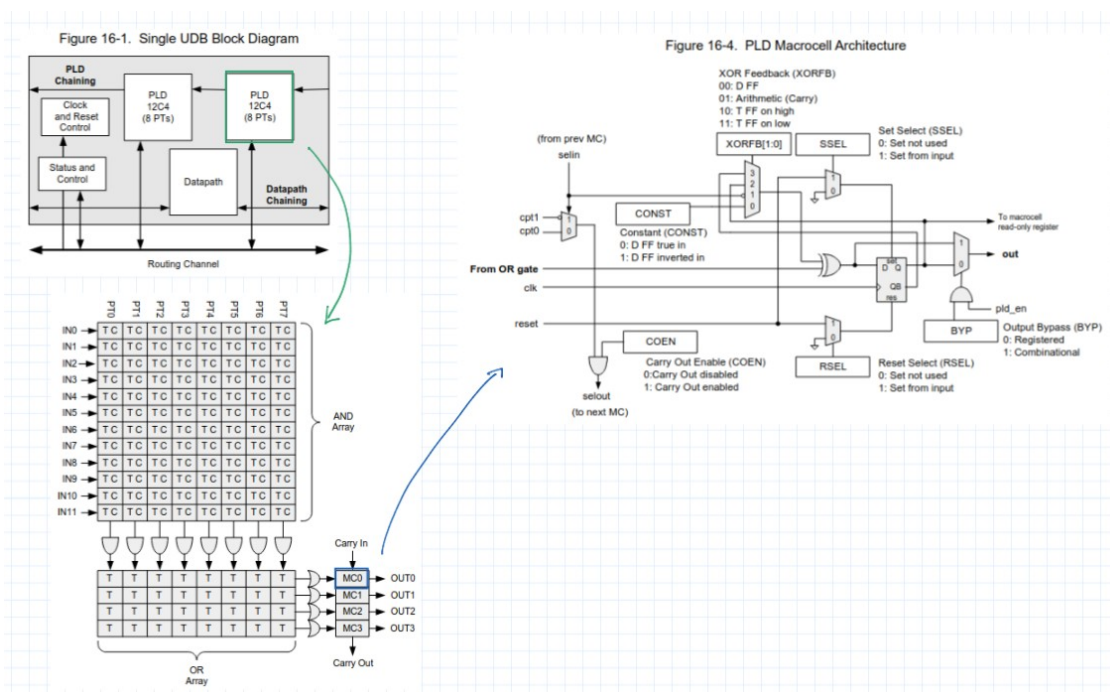
- Charlieplexing LEDs, debouncing switches, or controlling addressable LEDs using a SPI peripheral
- Zero-software ultrasonic distance sensing, quadrature decoding, or 7-segment display driver
- Complementary waveform generator with deadband control or 2-to-1 multiplexer
- Edge detector or PRNG
- Fast pulse detector, PWM steering, or glitch-free clock signal

Digital logic for all!

If you like the idea of configurable logic alongside your microcontroller but don't see yourself using a PIC16 or AVR device in the near future, there are still plenty of ways you can incorporate capabilities like this in your next project.

Cypress PSoC

Many (but not all) of the Cypress PSoC family of microcontrollers have configurable logic blocks called **universal digital blocks** (UDB).

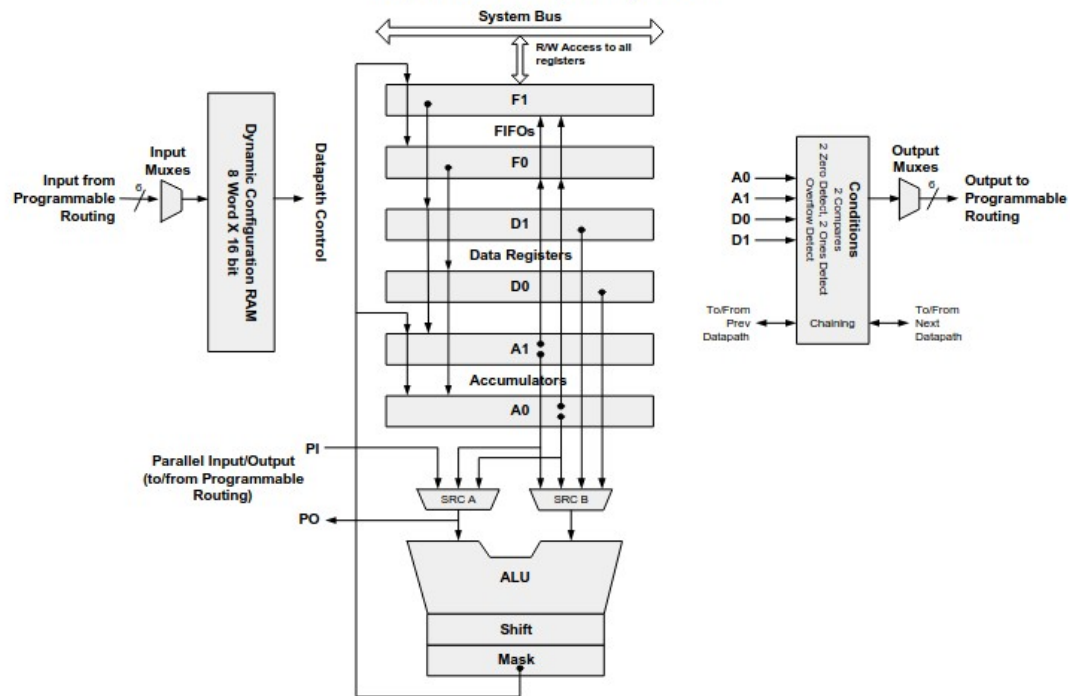


From the "PSoC 4 Architecture Technical Reference Manual"

The UDBs each contain two PLDs that are noticeably more capable than Microchip's CLB. The PLDs inside each UDB are formed from a 12-term AND-matrix followed by an 8-term OR-matrix (similar to the old [PLAs](#)), which is then routed to a macrocell. The macrocell can keep the output unregistered (i.e. combinational) or it can store the output of the OR gate in either a D or T flip-flop. The macrocell also implements carry logic between the macrocells as well as various reset functions.

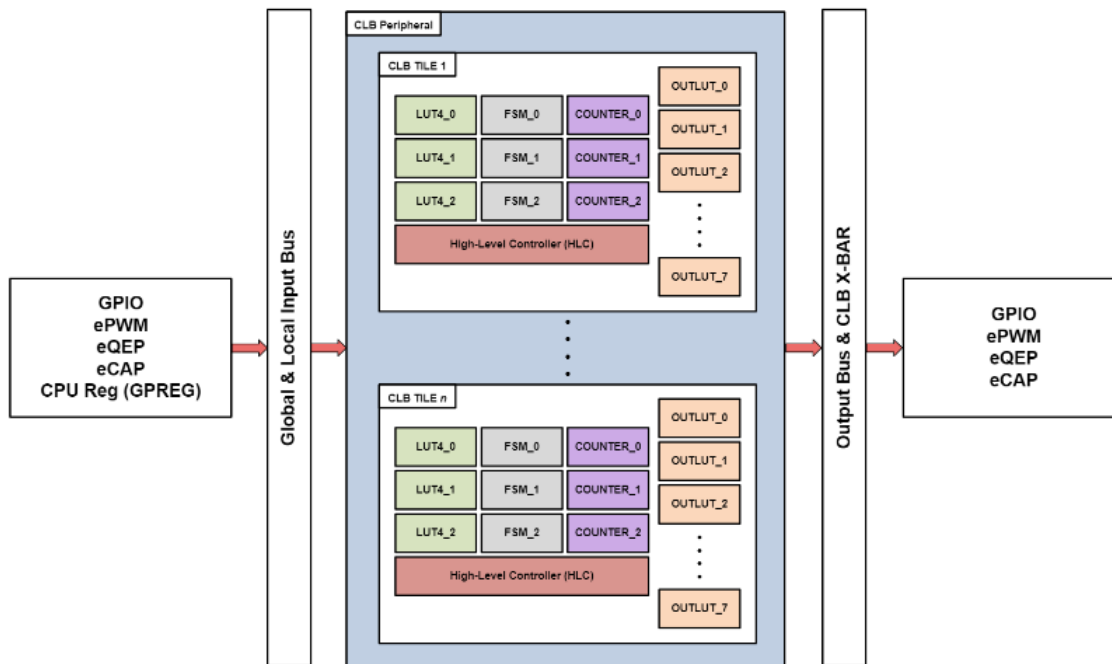
The "Datapath" element is an interesting addition, as it contains an ALU to more efficiently implement arithmetic functions.

Figure 16-6. Datapath Top Level



Texas Instruments C2000

The TI C2000 family of microcontrollers contain their own **configurable logic blocks**, that are also quite a bit more capable than the CLBs from Microchip.



From TI Resource Explorer: Configurable Logic Block (CLB)

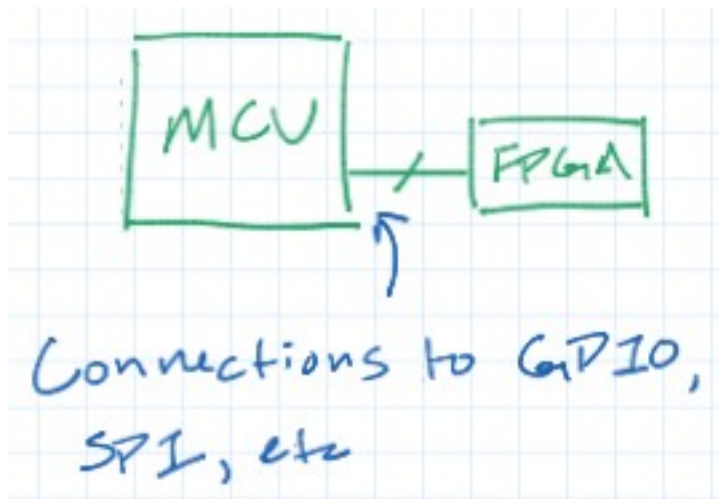
The CLB on a C2000 microcontroller contains up to 8 tiles, with each tile containing the following submodules:

- 3x 4-input LUTs
- 3x FSMs (each with up to 4 states)
- 3x 32-bit counters
- 8x 3-input output LUTs (these are what allow the CLB tiles to send data out to the rest of the system)
- 1x high-level controller
- 8x asynchronous output conditioning blocks

The inputs to each submodule can come from any of the other LUTs, FSMs, or counter submodules within that tile or from up to 8 other signals in the system, allowing for an enormous degree of configurability.

Field-programmable Gate Array (FPGA)

Of course, the ubiquitous piece of configurable digital logic is the FPGA, which we could always add to our microcontroller externally.



Everything that FPGAs are famous for (soft-core CPUs; fast, custom digital logic; etc) would be available to your embedded system, in that case! And it wouldn't necessarily need to be pricey, either: ice40 FPGAs with 48 logic elements can be had on DigiKey for [\\$2.40 in single quantities](#).

The main advantage to this approach is that we can now add configurable digital logic to *any* MCU. The main disadvantages are

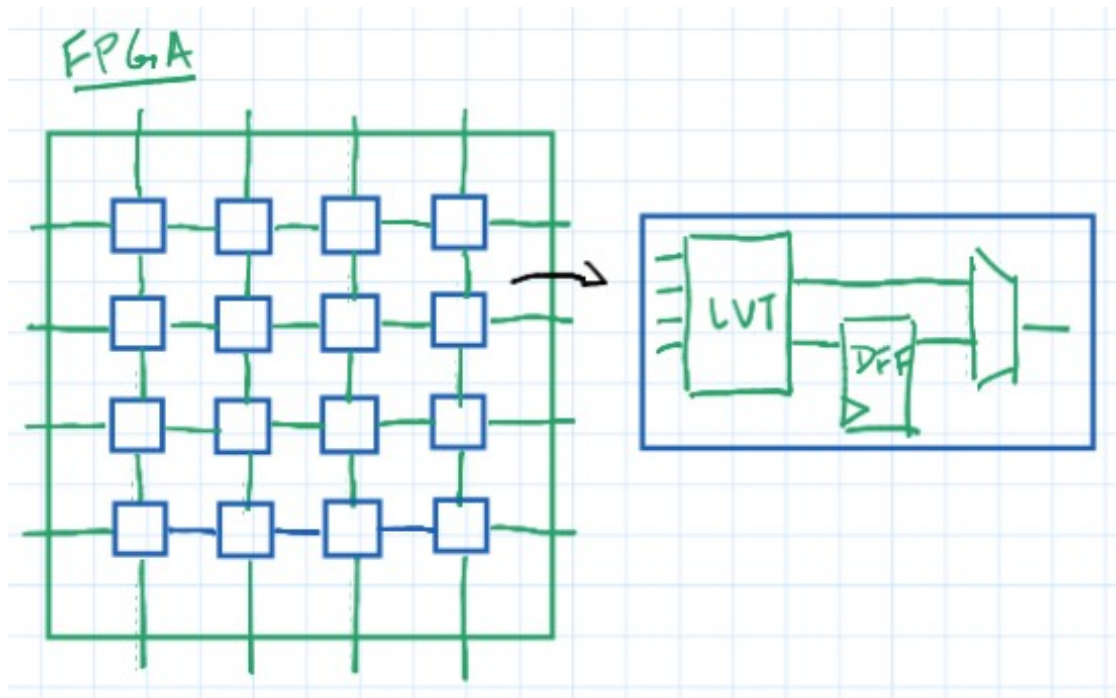
1. our system gets a bit more complex and
2. our digital logic doesn't have access to any internal MCU signals (such as interrupt lines or data/memory busses) and has to interface with it through the external pins on the MCU.

Complex programmable logic device (CPLD)

Another option for external programmable logic is a CPLD. CPLDs are more complex than a PLA (which are built using AND-OR matrices, like what we saw above inside the PLDs in the Cypress UDBs) and less complex than an FPGA. The main difference between CPLDs and FPGAs seems to be the ways in which

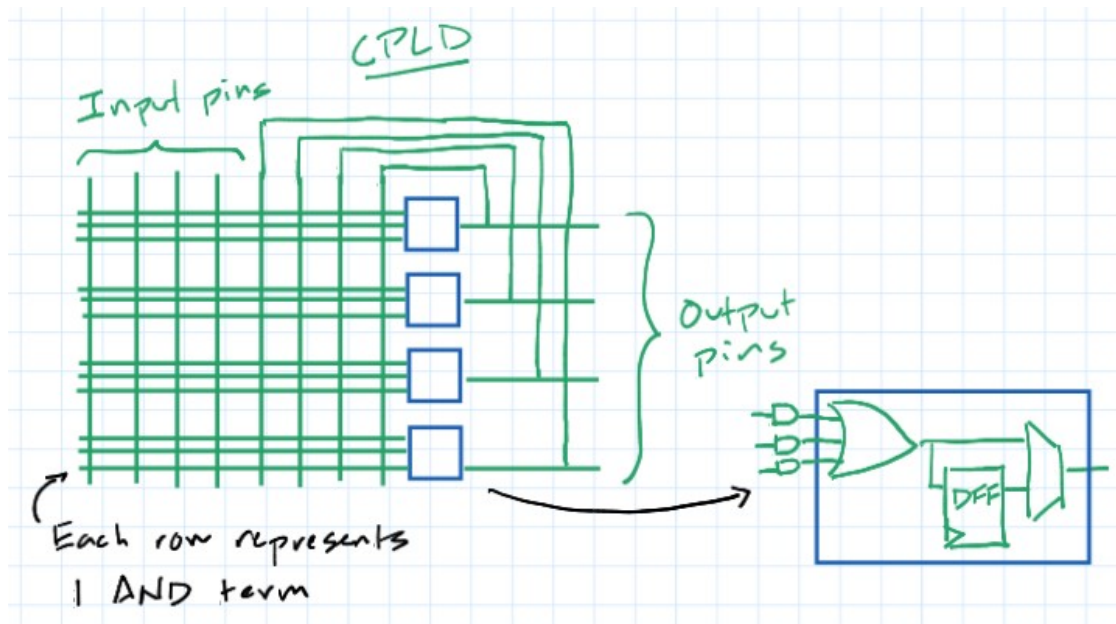
- the individual logic elements implement configurable logic and
- the way signals are routed to and between external pins and the individual logic elements.

For FPGAs, the individual logic elements are loaded at start-up from non-volatile memory with the LUT pattern that defines each block. Blocks are connected locally to each other, so signals that travel from one side of the FPGA to the other generally have to pass through other logic blocks along the way.



This makes FPGAs ideal for designs that need LOTS of logic elements and the higher capability that comes with that.

For CPLDs, on the other hand, individual logic elements (called "macrocells") use internal Flash memory or fuse bits to set the pattern of AND and OR terms that define the custom logic (though some do use LUTs, like FPGAs). The input pins on a CPLD and the outputs of all of the macrocells are connected to an enormous matrix which allows any of them to be used as inputs to each AND matrix (or LUT).



This makes CPLDs lower power and faster to start-up than FPGAs, with more predictable timing, also. CPLDs also do not require any external non-volatile memory, like FPGAs do, and typically require only a single voltage level to power them (FPGAs often require several). However, it also limits how many macrocells a CPLD could feasibly contain (the interconnect matrix would be infeasibly huge for CPLDs with several thousand macrocells). CPLDs are priced similarly to FPGAs; CPLDs containing 64 macrocells can be had on DigiKey for [\\$2.40 in single quantities](#).

The design space grows

Interestingly, all of the options above are quite a bit more powerful than the Microchip CLBs.

"Wait, I could build a laser tag system with the CLBs; what could I do with 32 macrocells/logic elements?!"

LOTS. For example, a [minimal 8-bit CPU](#). Imagine that: for \$2.40 you could add a small co-processor to your embedded system. Not bad, eh?

And now, let's put some of this new knowledge into practice!

Challenge Question

In what ways could the following systems benefit from a piece of configurable digital logic?

1. Industrial Motor Control Unit with Sensor Feedback and Safety Interlocks ([possible solutions](#))

1. This embedded system manages a small 3-phase BLDC motor used in an industrial conveyor belt. The system is controlled by a mid-range MCU which handles high-level functions like:
 1. Receiving commands via CAN bus or UART
 2. Controlling the motor driver IC (PWM generation)
 3. Monitoring position and speed sensors (hall or quadrature encoder)
 4. Enforcing safety interlocks (e.g., emergency stop, overcurrent protection, limit switches)
 2. Currently, the MCU performs the following in software:
 1. Quadrature decoding — ISR-driven edge detection from encoder A/B signals to track position and direction.
 2. Emergency stop FSM — A small state machine that reacts to various digital inputs (E-stop, fault, limit switch) to disable PWM outputs and signal fault conditions.
 3. PWM synchronization — MCU timer ISR ensures motor PWM updates only happen when all sensors are in a valid state.
 4. Input filtering/debouncing — Software debouncing of limit switches and fault inputs.
 5. Fault detection timing — If fault input remains active for >50 ms, trigger a system shutdown.
 3. Ex: A CLB/CPLD could perform quadrature decoding of the motor's Hall sensors, ensuring no missed counts even at high speeds.
2. Smart Sensor Hub for Industrial IoT Network ([possible solutions](#))
 1. This embedded device aggregates data from multiple industrial sensors (temperature, pressure, vibration) and communicates with a central controller using various serial protocols — UART, SPI, and an older custom interface (e.g., RS-485-based with a nonstandard timing). It's powered by a low-power MCU that is responsible for:
 1. Sampling sensor data
 2. Managing protocol conversions
 3. Handling time synchronization between multiple connected sensors
 4. Reporting data packets via Ethernet or LoRaWAN
 2. Initially, the MCU handles all the following tasks in firmware:

1. Bit-banged serial protocol decoding — For legacy sensors that use a custom serial interface with irregular timing (not supported by standard UART/SPI/I²C peripherals).
 2. Packet framing FSM — Software FSM builds and parses data packets for communication between sensors and the hub.
 3. Time-slot arbitration — Ensures that multiple sensors sharing a bus transmit in assigned time slots without collision.
 4. Signal synchronization — Aligns sensor sampling to an external clock pulse or trigger input.
 5. Edge timestamping — Software timestamps rising/falling edges from sensors (e.g., vibration pulse train) to compute frequency or duty cycle.
3. Ex: A CLB/CPLD could translate the custom serial protocol used by one of the sensors into SPI for the MCU

Conclusion

Adding a small piece of digital logic to your embedded system can be a very useful thing! Hardware implementations of functions like FSMs or Manchester encoding can operate much faster, be much more reliable, and operate at lower power than software implementations. My favorite way to do this is with the CLB/CLC/CCL peripherals from Microchip. Although they are less capable than other options, they are also simpler and still able to implement some very useful functions. Other ways to add configurable digital logic include using a Cypress PSoC microcontroller, a TI C2000 microcontroller, or using an external FPGA or CPLD. All four of those last options are quite a bit more complex (and also capable) than the Microchip CLB, though.

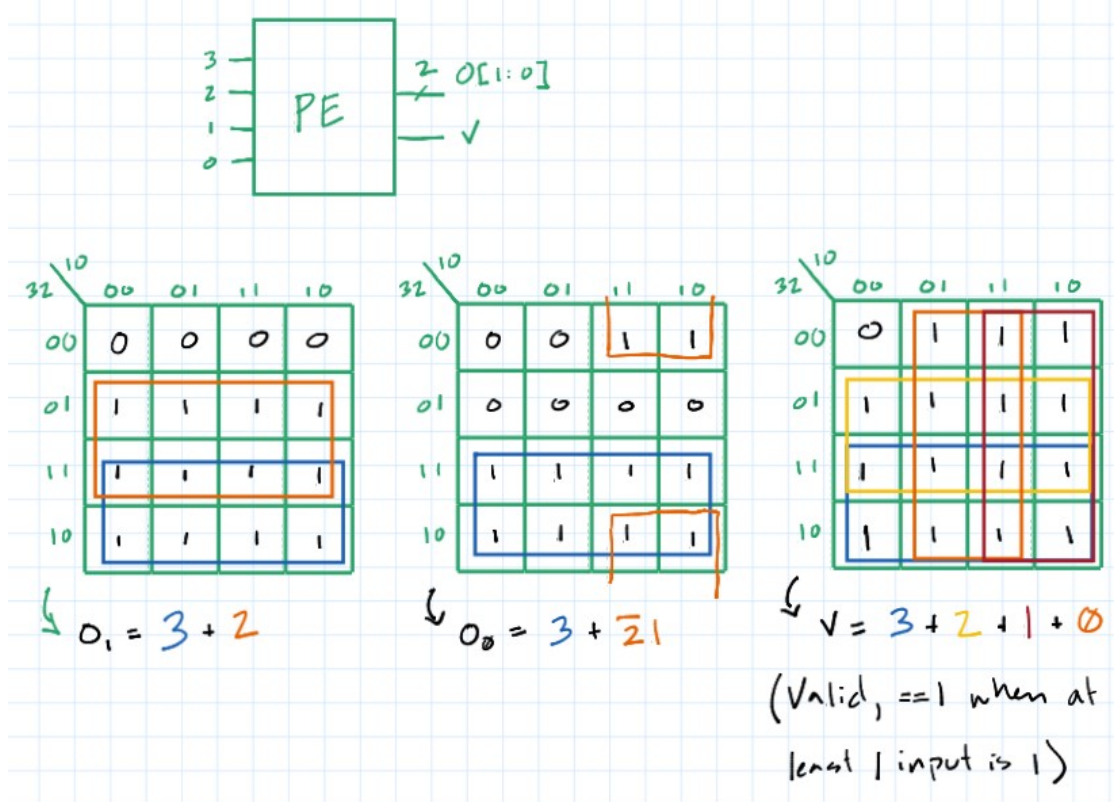
And I've one last **challenge question** for you!

List three or more things you've learned (or were reminded of) while reading this article.

If you've made it this far, thanks for reading and happy hacking!

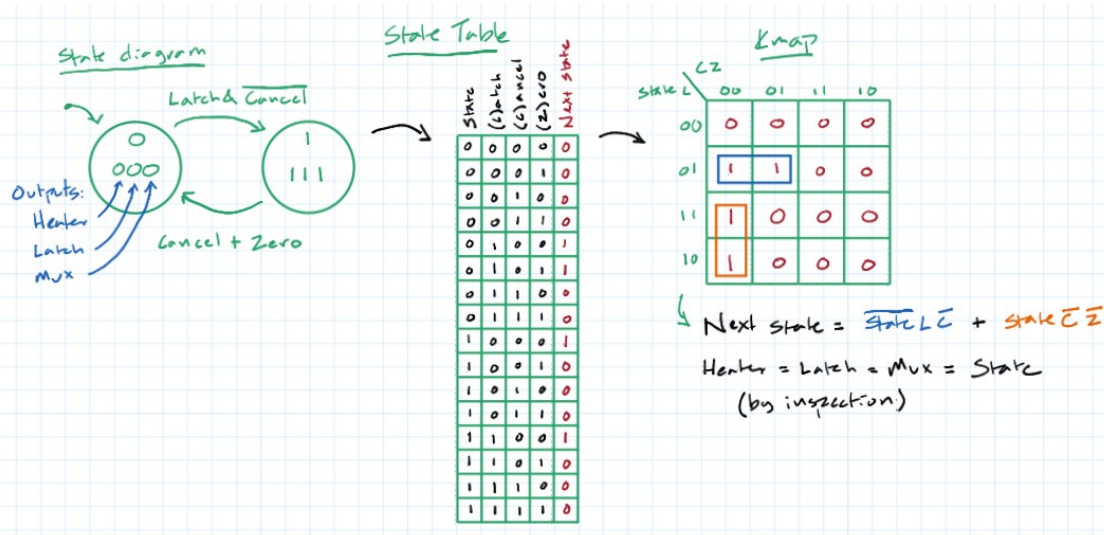
Solutions

Priority encoder



See it in simulation [here](#).

Toaster FSM



Motor control

Some things that a CLB/CPLD could do include:

- Decoding of hall sensors/quadrature encoder
- Safety FSM to:
 - Shut down motor when limit switches are reached
 - Send "system shutdown" message when e-stop is hit, current/temperature sensors are over a threshold, other fault conditions have persisted for some amount of time or have occurred more frequently than a set limit, etc.
- Deadband insertion on PWM signal
- Button debouncing
- Implementation of trapezoidal ramp up/down of motor speed
- PID control of motor speed, position
- CAN transceiver
- UART-to-CAN converter

Smart sensor

Some things that a CLB/CPLD could do include:

- Digital LPF of sensor data
- Conversion from older, custom serial protocol to UART, SPI, I2C, etc
- Conversion of all serial protocols to UART, SPI, I2C (unified serial interface for the MCU)
- Bus arbitration of multiple sensors on the same bus
- Calculate and verify checksum of incoming sensor data messages. Can alert MCU when valid packets arrive.
- External trigger synchronization (multiple sensors are triggered to sample simultaneously to a single signal from the MCU)
- Timestamping of sensor data or of rising/falling sensor signals (using an internal N-bit counter) to better correlate the timing relationship between sensor measurements
- Small FIFO of sensor data (to lessen the timing requirements on the MCU)